

Gantt Chart Summary

Here is a simplified Gantt chart outlining the timeline for the completion of the software from February 1st to May 15th:

Here is a summary of the tasks and their respective durations:

1. Prepare internal format

- Start Date: February 1st
- End Date: February 10th
- Duration: 10 days

2. Develop login and signup pages

- Start Date: February 11th
- End Date: February 20th
- Duration: 10 days

3. Create a database for different students

- Start Date: February 21st
- End Date: March 10th
- Duration: 18 days

4. Create attainment matrix

- Start Date: March 11th
- End Date: March 20th
- Duration: 10 days

5. Create Student Course Outcome (SCO)

- Start Date: March 21st
- End Date: April 5th
- Duration: 16 days

6. Add additional features

- Start Date: April 6th
- End Date: April 25th
- Duration: 20 days

7. Implement grades summary feature

- Start Date: April 26th

- End Date: May 15th
- Duration: 20 days

The total duration of the project from February 1st to May 15th is approximately 105 days, assuming that the year in question is a leap year (due to the inclusion of February 29th).

Please note that this is a simplified representation of the project timeline. In a real-world scenario, each task may have dependencies on other tasks, and there may be additional tasks such as testing, bug fixing, and deployment that are not accounted for in this timeline. Additionally, buffer times for unexpected delays or additional work may not be included in this timeline.

Task	Start Date	End Date	Duration
---	---	---	---
Task 1: Prepare internal format	2/1/2024	2/10/2024	10 days
Task 2: Develop login and signup pages	2/11/2024	2/20/2024	10 days
Task 3: Create a database for different students	2/21/2024	3/10/2024	19 days
Task 4: Create attainment matrix	3/11/2024	3/20/2024	10 days
Task 5: Create Student Course Outcome (SCO)	3/21/2024	4/5/2024	16 days
Task 6: Add additional features	4/6/2024	4/25/2024	20 days
Task 7: Implement grades summary feature	4/26/2024	5/15/2024	20 days

Gantt chart

INTRODUCTION

Introducing our software, the Teaching-Learning Attainment System with CO-PO Mapping and Automation. This innovative solution emerges as a response to the evolving educational landscape in our college and also for others, where the effective assessment of teaching-learning outcomes is imperative for institutional success.

Designed to streamline the assessment process in educational institutions, our software is meticulously crafted to address the challenges faced by educators in measuring Course Outcome to Program Outcome (CO-PO) mapping attainment. By automating manual tasks and leveraging advanced algorithms, the software empowers educators to enhance teaching effectiveness while reducing administrative burdens.

Problem Definition:

The problem we are addressing in this project is the cumbersome and error-prone process faced by professors when managing student attainments and grading using manual Excel sheets. This manual method is time-consuming, prone to mistakes, and lacks automation. Our goal is to develop a software solution that automates the student attendance and grading process, saving time for professors, reducing errors, and improving overall efficiency in managing student data and grades.

In addition to automating the student attainments and grading process, we are also addressing the challenge of tracking and assessing the attainment of Course Outcomes (CO), PEOs and Program Outcomes (PO). The current manual system makes it difficult to efficiently monitor and evaluate how well students are meeting the specified COs and POs. By incorporating CO and PO attainment tracking into our software, we aim to provide a comprehensive solution that not only streamlines grading but also enhances the assessment of student learning outcomes, ensuring better alignment with educational objectives and accreditation standards.

Home Insert Page Layout Formulas Data Review View								
Clipboard		Font			Alignment			
AG9								
	A	B	C	D	E	F	G	AA
1								
2	Mapping of MST-1							
3	Question/CO	CO1	CO2	CO3	CO4	CO5	CO6	
4	Q1	1						
5	Q2	1						
6	Q3				1			
7	Q4			1				
8	Q5							
9	Q6							
10		A1 CO1 =	A1 CO2 =	A1 CO3 =	A1 CO4 =	A1 CO5 =	A1 CO6 =	
11		83.8141	0	73.33333	73.33333	0	0	
12								
13								
14	Mapping of MST-II							
15	Question/CO	CO1	CO2	CO3	CO4	CO5	CO6	
16	Q1		1					
17	Q2					1		
18	Q3					1		
19	Q4						1	
20	Q5							
21	Q6							
22		A2 CO1 =	A2 CO2 =	A2 CO3 =	A2 CO4 =	A2 CO5 =	A2 CO6 =	
23		0	78.30189	0	0	68.88795	69.35484	
24								
25								
26	Mapping of MST-III							
27	Question/CO	CO1	CO2	CO3	CO4	CO5	CO6	
28	Q1							
29	Q2							
30	Q3							
MST1 MST2 MST3 ASS. TUT. External Award Attainment								

Excel sheet

PROJECT OVERVIEW:-

we can provide a detailed explanation of the software solution we have developed to automate student attendance, grading systems, and CO-PO attainment tracking for professors. Here's a comprehensive project overview:

Our project aims to revolutionize the way professors manage student data by introducing a software solution that automates the tedious tasks of taking attendance, calculating grades, and tracking Course Outcomes (CO) and Program Outcomes (PO). By leveraging technology, our software simplifies the process for professors, saving them valuable time and reducing the likelihood of errors commonly associated with manual Excel-based systems.

Through our software, professors can efficiently record student attendance, input grades, and generate reports with ease. Additionally, the incorporation of CO-PO attainment tracking allows for a more comprehensive evaluation of student learning outcomes, enabling educators to assess how well students are achieving the specified course and program objectives.

Overall, our project not only streamlines administrative tasks but also enhances the assessment of student performance, providing a holistic solution that aligns with educational goals and accreditation standards. By offering a user-friendly interface and robust functionality, our software aims to improve the efficiency and effectiveness of the grading process while promoting a data-driven approach to academic assessment.

Software specification

- 1) Accepting the values of CO-PO mapping, target attainment, and direct & indirect assessment to the database.
- 2) Calculation of course outcome & program outcome attainment.
- 3) Displaying the CO & PO attainment on different screens.

LITERATURE SURVEY

The development of software solutions for educational assessment and attainment tracking has been a subject of significant research and development in recent years. Various studies have explored the challenges faced by educators in measuring and assessing teaching-learning outcomes, as well as the potential benefits of implementing automated systems in educational settings.

Existing Systems:

Eklavya –Process Automation Platform for NBA accreditation is designed and developed in such a way that it simplifies the accreditation process of the institute in a better way with accurate CO-PO mapping results along with tracking of student performance.

MasterSoft.com : Simplify COs, PEOs, and POs/PSOs Attainment with Detailed Outcome Mapping. Track, monitor, and accurately map student's academic performance With CO-PO mapping & assessments for continuous outcome improvement. Understand exact challenges faced by students through feedback & by assessing their learning progress.

Other Existing systems in the context of teaching and learning attainment can vary depending on the specific domain or field of study. However, there are a few common types of systems that are widely used:

1. Learning Management Systems (LMS): LMS platforms, such as Moodle, Canvas, and Blackboard, are widely used in educational institutions. They provide a centralized platform for managing course content, assignments, assessments, and communication between teachers and students.

2. Assessment and Grading Systems: These systems focus on automating the assessment and grading process. They often include features such as online quizzes, automated grading, and performance analytics. Examples include GradeScope, Turnitin, and ExamSoft.

3. Learning Analytics Systems: These systems collect and analyze data on student learning behaviors and outcomes. They use techniques such as data mining and machine learning to provide insights into student performance and engagement. Examples include Brightspace Analytics, SNAPP, and Learnalytics.

4. Student Information Systems (SIS): SIS platforms, like PowerSchool and Infinite Campus, are used to manage student data, including enrollment, attendance, grades, and transcripts. They provide a comprehensive view of student information for administrators, teachers, and parents.

5. Personalized Learning Systems: These systems aim to tailor the learning experience to individual students' needs and preferences. They often use adaptive learning algorithms to deliver personalized content and assessments. Examples include Knewton, DreamBox, and Khan Academy.

PROPOSED SYSTEM:-

The proposed system is developed utilizing Python programming language (with Pandas, Seaborn and Numpy libraries) and DBMS. It comprises two primary sections: data insertion and attainment calculation. The User Interface is tailored to accommodate both staff and admin functionalities. Administrators possess privileges to upload, modify, and remove student lists pertinent to the current academic year. Meanwhile, staff members are empowered to input data such as CO-PO matrices, target values, as well as direct and indirect assessment metrics.

Upon validation of all input values, the system automatically generates a comprehensive CO and PO attainment sheet, providing insights into the achievement status of these outcomes. This information serves as a valuable resource in curriculum design and refinement. The CO-PO attainment calculation plays a pivotal role in evaluating Outcome-Based Education (OBE), facilitating iterative enhancements to program outcomes and course objectives.

Courses within the curriculum are delivered through diverse teaching-learning methodologies, including traditional classroom lectures, hands-on laboratory experiments, interactive workshops, guest lectures, and project-based learning

experiences. The final attainment is determined through a combination of Direct and Indirect assessments. Direct assessments encompass various components such as term tests, end-semester examinations, assignments, laboratory work, and project outcomes. Indirect assessments, on the other hand, typically involve gathering feedback through course exit surveys administered at the culmination of each semester.

Tools Used:

Python:

Python is a popular programming language known for its simplicity and readability. It is widely used in various fields such as web development, data science, artificial intelligence, and automation. Here's a thorough explanation of Python:

1. **Easy to Learn:** Python's syntax is clear and easy to understand, making it a great language for beginners. It emphasizes readability and uses indentation to define code blocks, which helps maintain clean and organized code.
2. **Interpreted Language:** Python is an interpreted language, meaning that the code is executed line by line by the Python interpreter. This allows for quick development and testing without the need for compilation.
3. **Versatile:** Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming. This versatility makes it suitable for a wide range of applications.
4. **Rich Standard Library:** Python comes with a vast standard library that provides ready-to-use modules and packages for various tasks, such as working with files, networking, and data processing. This extensive library reduces the need to write code from scratch for common functions.
5. **Community Support:** Python has a large and active community of developers who contribute to its growth and development. This community support means that there

are numerous resources, libraries, and frameworks available to help with different projects.

6. Data Science and Machine Learning: Python is widely used in data science and machine learning due to libraries like NumPy, Pandas, and Scikit-learn. These libraries provide powerful tools for data manipulation, analysis, and machine learning model development.

7. Web Development: Python can be used for web development through frameworks like Django and Flask. These frameworks simplify the process of building web applications by providing tools for handling routing, templates, databases, and more.

8. Automation: Python is commonly used for automation tasks, such as scripting repetitive actions, interacting with files and databases, and controlling hardware devices. Its ease of use and readability make it ideal for writing automation scripts.

Overall, Python's simplicity, versatility, and strong community support make it a popular choice for developers across various domains. Whether you're a beginner or an experienced programmer, Python offers a straightforward and powerful language for a wide range of applications.

Numpy:

Numpy is like a powerful library in Python for numerical computing. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently. Numpy is widely used in data science, machine learning, and scientific computing due to its speed and versatility in handling numerical operations. It serves as a fundamental building block for many other Python libraries and tools used in data analysis and computation.

Pandas:

Pandas is like a super useful library in Python for data manipulation and analysis. It offers data structures like DataFrames that make it easy to work with structured data, such as tables with rows and columns. Pandas allows you to load, clean, transform,

and analyze data efficiently. It provides functionalities for handling missing data, merging datasets, grouping data, and performing statistical operations. Pandas is widely used in data science, machine learning, and data analysis projects to explore and process data effectively.

Feasibility Study

Feasibility for building the Teaching-Learning Attainment System with CO-PO Mapping and Automation:

1. Requirement Analysis:

- a. Conduct interviews and surveys with stakeholders (educators, administrators, students) to understand their needs and expectations from the software.
- b. Document the specific requirements, features, and functionalities desired in the software.

2. Economic Feasibility:

- a. Calculate the cost of developing the software, including initial development expenses, maintenance costs, and potential licensing fees.
- b. Estimate the potential cost savings and benefits that will result from using the software, such as reduced labor hours, improved efficiency, and enhanced data accuracy.
- c. Conduct a cost-benefit analysis to determine if the financial investment in the software aligns with the expected returns and benefits for educational institutions.

3. Design Phase:

- a. Create a detailed design of the software architecture, including database schema, user interface layout, and system workflow.
- b. Define the algorithms and methodologies for CO-PO mapping attainment calculation and automation features.

4. Development:

- a. Implement the software based on the design specifications using Python programming language with Pandas, Numpy and Seaborn.
- b. Develop user interfaces for staff and admin functionalities, ensuring usability and accessibility.
- c. Integrate algorithms for CO-PO mapping attainment calculation, automated grading, feedback generation, and tracking functionalities.

5. Testing:

- a. Perform rigorous testing of the software to identify and rectify any bugs or errors.
- b. Conduct unit testing, integration testing, and user acceptance testing to ensure the software meets the requirements and functions correctly.
- c. Gather feedback from users during the testing phase to make necessary improvements.

6. Deployment:

- a. Deploy the software in a test environment initially to validate its performance and functionality.
- b. Once validated, deploy the software in the production environment, ensuring proper configuration and setup.
- c. Provide training to users (staff, admin) on how to use the software effectively.

Later, we gather the feedbacks from the professors.

SYSTEM ANALYSIS & DESIGN

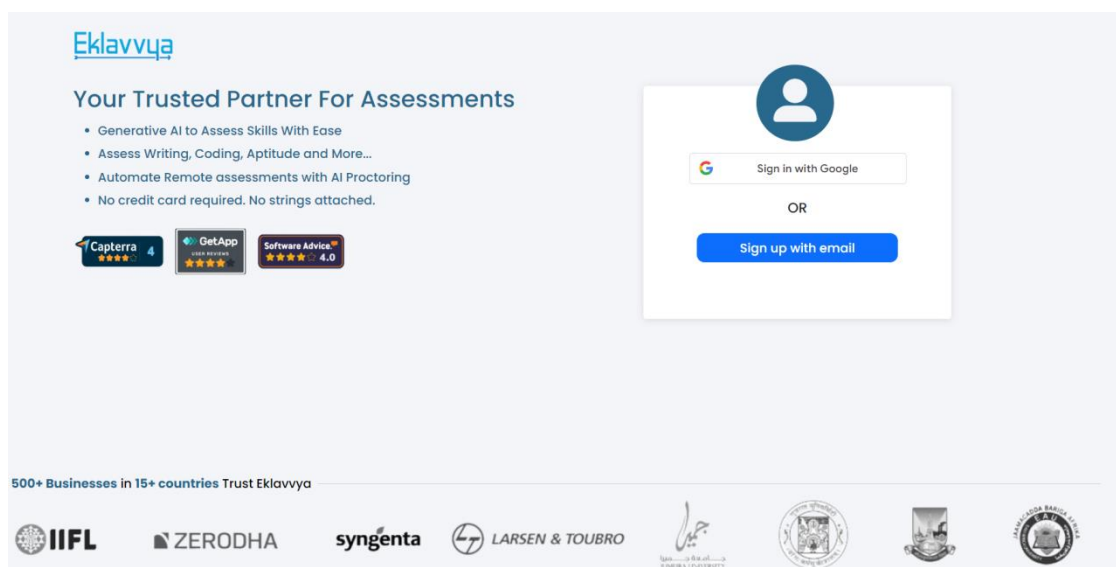
System Analysis:

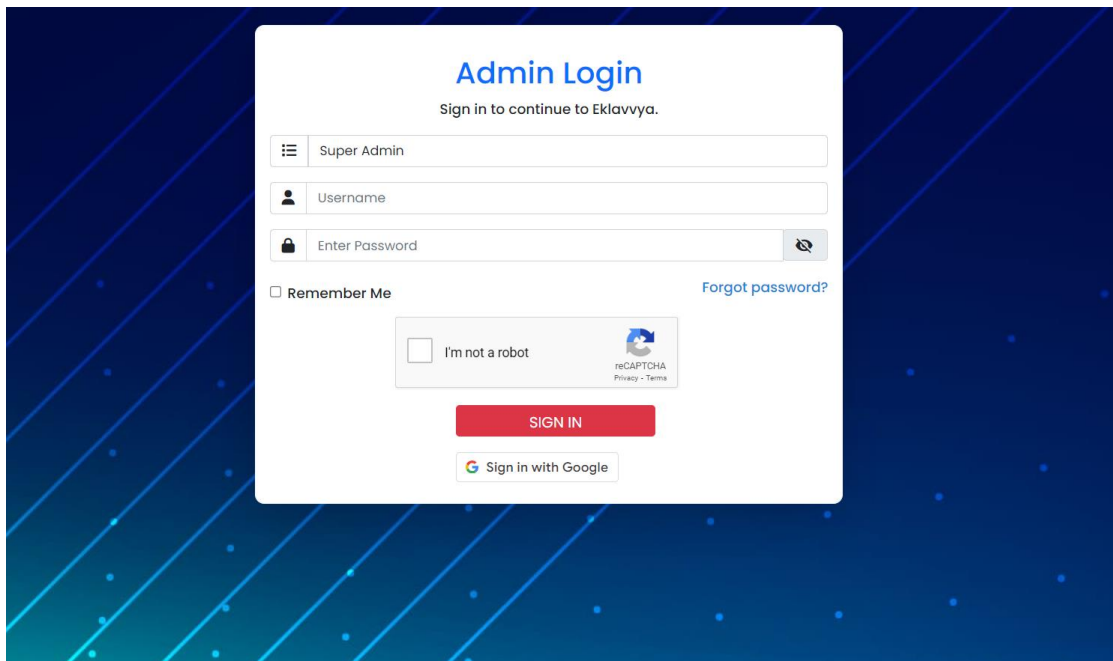
System analysis is like taking a closer look at the software project to understand what it needs to do and how it should work. It involves evaluating the current system, figuring out what users want, and defining what the software should do.

During system analysis, we identify the functions the software needs to perform, both basic tasks like logging in and more complex ones like generating grades summaries. We also consider non-functional aspects such as security, speed, and reliability.

Current System Evaluation:

Reviewing the existing system “Eklavya-process automation platform for NBA” components such as the login and signup pages, student database, attainment matrix, SCO, additional features, and grades summary. Assessing the functionality, usability, and performance of each component to identify strengths and areas for improvement.





Problem of the existing software

Much complex interface for the freshers or new professors.

Does not have a feature of downloading the file of students' data, etc.

It has not any kind of technique to change the formula of assessment when the professors want.

It requires internet connectivity to use.

Solutions and the aim of the our software

Need of building a simple interface for the professors.

Adding feature of exporting the data in excel as it can be helpful.

Making application flexible as much as changes in formulas and systems are easy to make.

Making dynamic databases for making the process of insertion and deletion of data more easier.

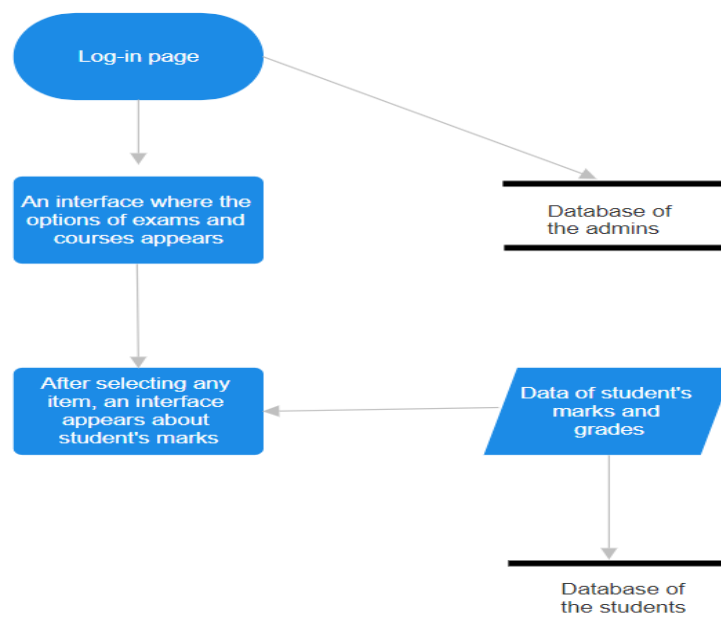
Design:

Design is like planning how the software will look and work based on the system analysis. It involves creating the layout, structure, and features of the software to meet the user's needs and make it easy to use. Design includes things like deciding where buttons go, how information is displayed, and how users interact with the software. Good design makes the software intuitive, visually appealing, and efficient to use.

Flowcharts:

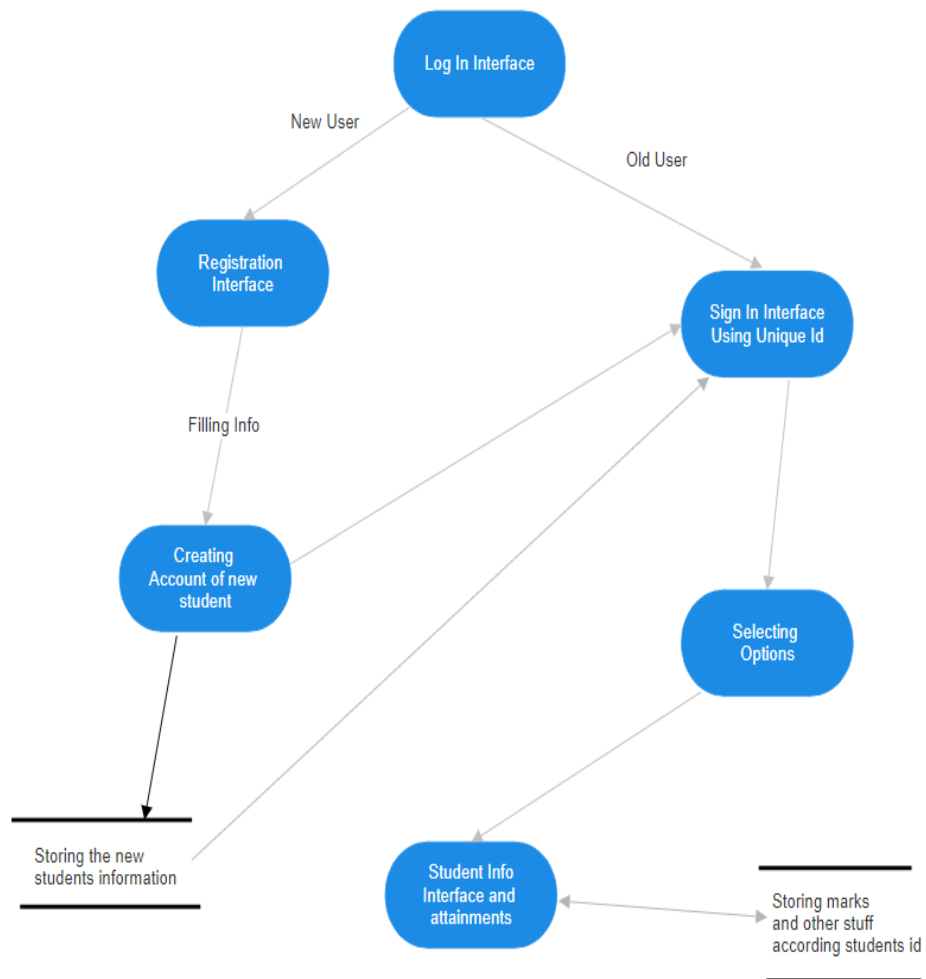
Flowcharts are like visual diagrams that show the steps in a process or system using symbols and arrows. They help to illustrate the flow of information or activities from start to finish. Flowcharts are commonly used in software development to represent algorithms, workflows, or decision-making processes in a clear and easy-to-understand way.

First and basic flowchart of the proposed software:



DFD LEVEL 2

Detailed Dataflow diagram:



DFD LEVEL 3

RESULTS / OUTPUTS

Here the results or interfaces look like:

- Registration form:



New Account Registration

Registration Form

Enter Email Id

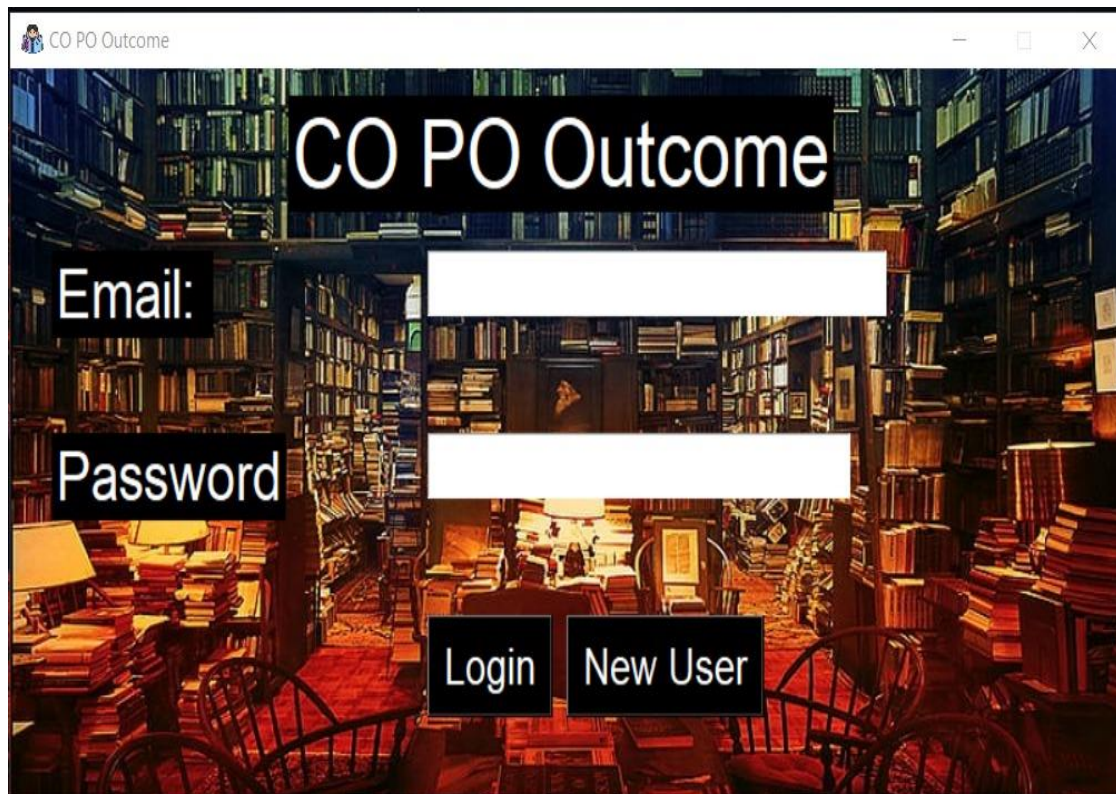
Enter Password

Enter Name

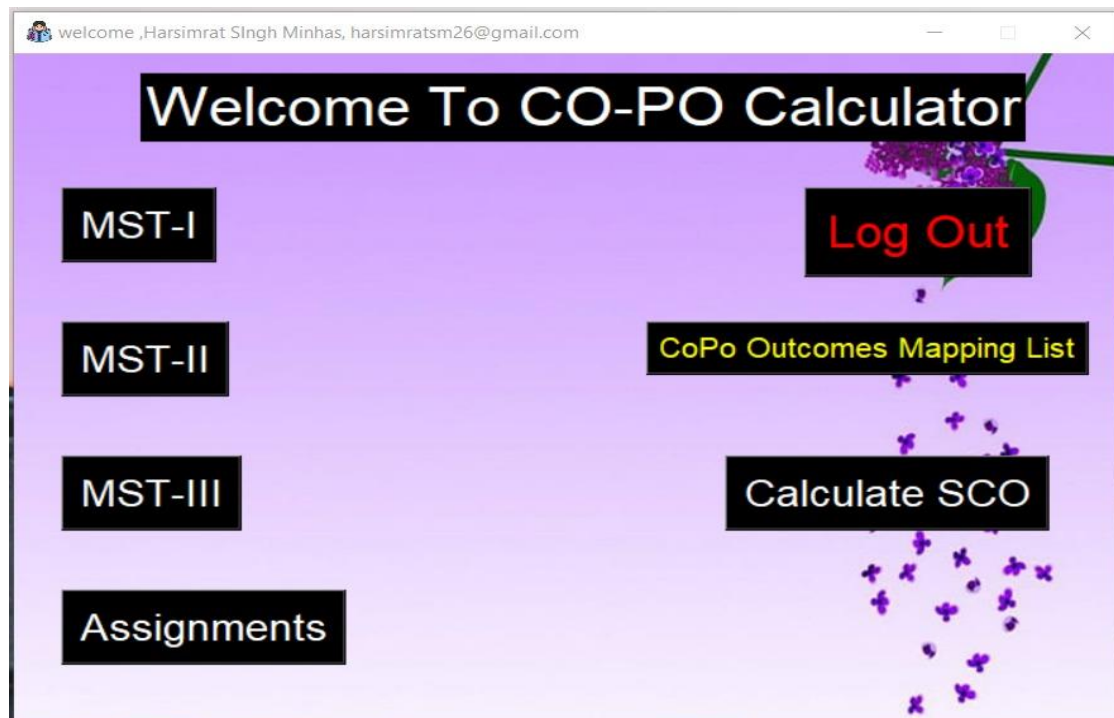
Enter Mobile

Create Account

- Log In interface:



- Selection Features:



- Student info section:

The screenshot shows a web application interface with a dark background. At the top right, there is a red square button with a white 'X' icon. Below this, the form contains several input fields and buttons. On the left, the labels 'Roll No:', 'Student Name:', 'Q1:', 'Q2:', 'Q3:', 'Q4:', 'Q5:', and 'Q6:' are listed vertically. To the right of each label is a white rectangular input field. Below the input fields, there are three buttons: 'Add', 'Clear', and 'Calculate'. The 'Add' and 'Clear' buttons are positioned side-by-side, and the 'Calculate' button is centered below them. To the right of the input fields, there is a button labeled 'matrix'.

Roll No:

Student Name:

Q1:

Q2:

Q3:

Q4:

Q5:

Q6:

REFERENCES

- [1]S. Amirtharaj, G. Chandrasekaran, K. Thirumoorthy, and K. Muneeswaran, “A Systematic Approach for Assessment of Attainment in Outcome-based Education,” *Higher Education for the Future*, vol. 9, no. 1, pp. 8–29, Dec. 2021, doi: <https://doi.org/10.1177/23476311211017744>.
- [2]A. Rajak, A. Shrivastava, and A. Tripathi, “An Approach to Evaluate Program Outcomes and Program Educational Objectives through Direct and Indirect Assessment Tools,” *International Journal of Emerging Technologies in Learning (iJET)*, vol. 14, no. 23, pp. 85–97, Dec. 2019, Available: <https://www.learntechlib.org/p/217241/>